

Открытая олимпиада школьников (информатика) 2025-26

Решения заданий заключительного этапа для 10 и 11 класса (типовой вариант)

1. Кодирование информации. Системы счисления (2 балла)

[Два умножения]

Про некоторое положительное вещественное число, меньшее 1, известно следующее:

1. Если это число умножить на 2_{10}^{100} , результат окажется целым числом.
2. Если это число умножить на 9_{10} и перевести в запись в двоичную систему, окажется, что эта запись числа не содержит значащих нулей.

Сколько существует таких чисел?

В ответе укажите целое число. Если таких чисел не существует или их бесконечно много, укажите в ответе NULL.

Примечание: число 0.1_2 содержит значащий ноль.

Пример ввода ответа:

17

Ответ: 50

Решение

Обратим внимание, что в соответствии с первым условием такое число содержит не более 100 значащих разрядов после запятой. Умножение на 9_{10} — это умножение на 1001_2 . Тогда эту арифметическую операцию можно представить как умножение исходного числа на $8_{10} = 1000_2$ и сложение результата с исходным числом, то есть сдвиг двоичной записи числа на 3 разряда влево и сложение с исходным числом. Тогда, чтобы в результате последней операции (сложения с исходным числом) получились только единичные разряды в двоичной записи, после сдвига всем разрядам исходного числа должны соответствовать противоположные (для «0» — «1», а для «1» — «0») значения разрядов с такими же индексами в сдвинутой записи. А значит, дробная часть исходного числа должна содержать чередование последовательностей из трех единиц и трех нулей.

Эта последовательность должна начинаться сразу после разделительной запятой, например, 0.111_2 , 0.111000111_2 , 0.111000111000111_2 и т.д. или отстоять от неё на 1 или 2 значащих нуля:

- 0.0111_2 , 0.0111000111_2 , 0.00111000111000111_2 и т.д.
- 0.00111_2 , 0.00111000111_2 , 0.00111000111000111_2 и т.д.

Например:

$$0.0111000111_2 \cdot 1001_2 = 11.1000111_2 + 0.0111000111_2 = 11.111111111_2.$$

В любом другом случае после сдвига на 3 разряда влево окажется хотя бы один индекс разряда, для которого и в сдвинутом, и в исходном числе окажутся одинаковые значения. Например, число 0.000111_2 после умножения на 1001_2 даст результат 0.11111_2 , который будет содержать только единицы в дробной части, но будет содержать значащий ноль в целой части, а значит, не подходит под условие.

Следовательно, нужно посчитать количество чисел, содержащих не более 100 разрядов после запятой в двоичной записи, таких, чтобы эти разряды образовывали последовательность с чередованием трех единиц и трех нулей и не имели либо имели 1 или 2 значащих нуля между делителем и этой последовательностью. Легко подсчитать, что таких последовательностей может быть ровно $17 + 17 + 16 = 50$, что и будет ответом на задание.

2. Кодирование информации. Объём информации (1 балл)

[Соты]

Петя пишет систему управления роботом, который перемещается по плоскости с гексагональной разметкой — правильными шестиугольниками, примыкающими друг к другу одним из ребер. Каждая команда программы перемещает робота на смежный шестиугольник в одном из шести возможных направлений. Петя решил записывать программу в память робота как последовательность кодов команд, используя для записи каждой команды минимально возможное, одинаковое для всех команд количество бит. Петя посчитал, что тогда в сегмент памяти, который у него есть, можно поместить программу длиной ровно 4096 команд.

Вася обратил внимание на такую особенность робота Пети — любая его программа всегда имеет количество команд, кратное 128. Он предложил рассматривать программу робота как последовательность подпрограмм, длиной 128 команд, кодируя каждую такую подпрограмму минимально возможным, одинаковым для всех возможных подпрограмм количеством бит.

Определите максимальную длину программы (кратную 128), которую можно записать в сегмент памяти, кодируя по методу Васи. В ответе укажите целое число.

Пример ввода ответа:
512

Ответ: 4736

Решение

Обозначим максимальную длину программы, помещающуюся в память при способе кодирования Пети, за A , а длину подпрограммы при способе кодирования Васи за B .

Поскольку количество различных направлений (и различных команд) равно 6, для хранения одной команды понадобится $\lceil \log_2 6 \rceil = 3$ бит. Тогда программа, закодированная способом Пети, будет занимать $3A$ бит, и это будет размером имеющегося сегмента памяти.

У Васи всего будет 6^B вариантов различных подпрограмм из B команд, и одна подпрограмма займёт в памяти $\lceil \log_2 6^B \rceil = \lceil B \log_2 6 \rceil$ бит.

Тогда всего Вася сможет записать в имеющийся сегмент памяти максимум $\left\lfloor \frac{3A}{\lceil B \log_2 6 \rceil} \right\rfloor$ подпрограмм длины B , и длина такой программы будет равна $\left\lfloor \frac{3A}{\lceil B \log_2 6 \rceil} \right\rfloor \cdot B$ командам.

Например, если $A = 4096$ и $B = 128$, то:

$$\left\lfloor \frac{3A}{\lceil B \log_2 6 \rceil} \right\rfloor \cdot B = \left\lfloor \frac{3 \cdot 4096}{\lceil 128 \log_2 6 \rceil} \right\rfloor \cdot 128 = \left\lfloor \frac{12288}{330} \right\rfloor \cdot 128 = 4736.$$

3. Основы логики (2 балла)

[Переусложнили...]

Обозначим за X некоторое натуральное число, записанное с помощью $n + 1$ бит, т.е. $X = (x_n x_{n-1} \dots x_0)_2$. Например, если $n = 3$ и $X = 0101_2$, то $x_3 = 0, x_2 = 1, x_1 = 0, x_0 = 1$.

Даны логические функции $A(X)$ и $B(X)$:

$$A(X) = \bigwedge_{i=0}^{n-1} (x_i \rightarrow x_{i+1}) \wedge (x_n \rightarrow x_0)$$

$$B(X) = \bigvee_{i=0}^{\lfloor \frac{n-1}{2} \rfloor} \neg \left(M(x_i, x_{i+\lceil \frac{n+1}{2} \rceil}, 1) \equiv M(x_i, x_{i+\lceil \frac{n+1}{2} \rceil}, 0) \right)$$

Здесь $\lfloor a \rfloor$ — значение a с округлением вниз, $\lceil a \rceil$ — значение a с округлением вверх.

Функция $M(a, b, c)$ задана таблицей истинности:

a	b	c	$M(a, b, c)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Сколько существует чисел X таких, что $A(X) = B(X)$, если $n = 19$? В ответе введите целое положительное число.

Примечание: битовая последовательность может начинаться с нуля.

Пример ввода ответа:

17

Ответ: 1022

Решение

Рассмотрим функцию $A(X)$. Она представляет собой конъюнкцию импликаций и равна 1 только в том случае, когда все импликации равны истине. Это значит, что не должно встречаться импликаций вида $1 \rightarrow 0$, и в

битовой последовательности X не должно быть последовательности 10. Последний член конъюнкции дополнительно накладывает ограничение, что не должно быть ситуации, когда $x_n = 1$ и $x_0 = 0$. То есть если $A(X) = 1$, то последовательность равна либо 000...000, либо 111...111, где общее количество цифр равно $n + 1$. В противном случае $A(X) = 0$.

Рассмотрим функцию $B(X)$. Используемая в ней функция $M(a, b, c)$ — функция большинства. Из неё видно, что $M(x_i, x_{i+\lceil \frac{n+1}{2} \rceil}, 1) = x_i \vee x_{i+\lceil \frac{n+1}{2} \rceil}$, а $M(x_i, x_{i+\lceil \frac{n+1}{2} \rceil}, 0) = x_i \wedge x_{i+\lceil \frac{n+1}{2} \rceil}$. Заменяя отрицание эквиваленции на исключающее "ИЛИ", получим:

$$B(X) = \bigvee_{i=0}^{\lfloor \frac{n-1}{2} \rfloor} \left(x_i \vee x_{i+\lceil \frac{n+1}{2} \rceil} \right) \oplus \left(x_i \wedge x_{i+\lceil \frac{n+1}{2} \rceil} \right) = \bigvee_{i=0}^{\lfloor \frac{n-1}{2} \rfloor} \left(x_i \oplus x_{i+\lceil \frac{n+1}{2} \rceil} \right)$$

Из этой формулы видно, что функция $B(X)$ работает с "половинами" битовой последовательности и равна 0 только тогда, когда "половины" битовой последовательности совпадают (например, 110110).

Если $A(X) = B(X) = 1$, то последовательность равна либо 000...000, либо 111...111, но при этом имеет разные "половины". Здесь мы получаем противоречие, и значит, что вариант с $A(X) = B(X) = 1$ не подходит, и эти две последовательности также не подходят.

Далее рассмотрим два случая для разной чётности значения n при условии $A(X) = B(X) = 0$.

Если n нечётно, то длина последовательности чётна, и сама последовательность делится ровно на две части. Первая половина последовательности ($\frac{n+1}{2}$ элементов) задаётся произвольно, а вторая половина равна первой. Тогда существует $2^{\frac{n+1}{2}}$ вариантов этой последовательности. В них входят две последовательности, описанные выше, они нам не подходят. Тогда в итоге будет $2^{\frac{n+1}{2}} - 2$ варианта последовательности.

Если n чётно, то длина последовательности нечётна, и центральный элемент не входит ни в первую половину, ни во вторую. Первая половина последовательности ($\frac{n}{2}$ элементов) задаётся произвольно, а вторая половина равна первой. При этом центральный элемент также произволен. Тогда существует $2^{\frac{n}{2}+1}$ вариантов этой последовательности. В них входят две последовательности, описанные выше, они нам не подходят. Тогда в итоге будет $2^{\frac{n}{2}+1} - 2$ варианта последовательности.

4. Кодирование информации. Алгоритмы обработки кодированной информации (1 балл)

[Взломщик поневоле]

Вася изучает алгоритмы шифрования и решил придумать свой собственный алгоритм. Он решил, что его алгоритм будет рассчитан на шифрование слов из букв современного латинского алфавита. Для каждой буквы в тексте Вася применяет следующие шаги:

1. Выяснить номер этой буквы в алфавите (Вася пронумеровал буквы от 1 до 26).
2. Выяснить номер в алфавите i -й буквы ключа.
3. Перемножить два данных числа - это и есть Васин код для буквы текста.
4. Увеличить i на 1 и взять по модулю длины ключа.

Исходно $i = 0$, а нумерация букв в ключе и в тексте для шифрования начинается с нуля.

Чтобы проверить свой алгоритм, Вася взял фразу «the quick brown fox jumps over the lazy dog», убрав из неё пробелы, повторил её 15 раз подряд и зашифровал полученный текст.

В результате он получил следующую последовательность чисел (см. [файл по ссылке](#)). Потом Вася решил проверить, можно ли расшифровать результат и понял, что потерял ключ. Единственное, что он помнит: все символы в ключе были различными. Помогите Васе выяснить, каким был ключ. В ответе введите последовательность строчных букв латинского алфавита без пробелов. Если же восстановить ключ невозможно, укажите в качестве ответа NULL.

Пример ввода ответа:

abcdef

Ответ: skjhbweryugqfptacxzd

Решение

Основная идея задачи: взять получившуюся последовательность чисел (из файла), перевести кодовую фразу в набор чисел и поделить числа из последовательности на числа, полученные из кодовой фразы. Так как буквы в ключе различны, вычисление останавливается в тот момент, когда появляется уже имеющаяся в ключе буква.

Проще всего решить эту задачу программно:

```
from itertools import cycle
```

```
with open('file.txt') as file:
    encoded = [int(x) for x in file.read().split()]
```

```

print(encoded)

phrase = [ord(ch) - ord('a') + 1 for ch in 'thequickbrownfoxjumpsoverthelazydog'] * 15
print(phrase)

key = []

for encoded_char, phrase_char in zip(encoded, cycle(phrase)):
    key_char = encoded_char // phrase_char

    if key and key[0] == key_char:
        print('Found a key')
        break

    key.append(key_char)
    print(encoded_char, phrase_char, key_char, sep='\t')

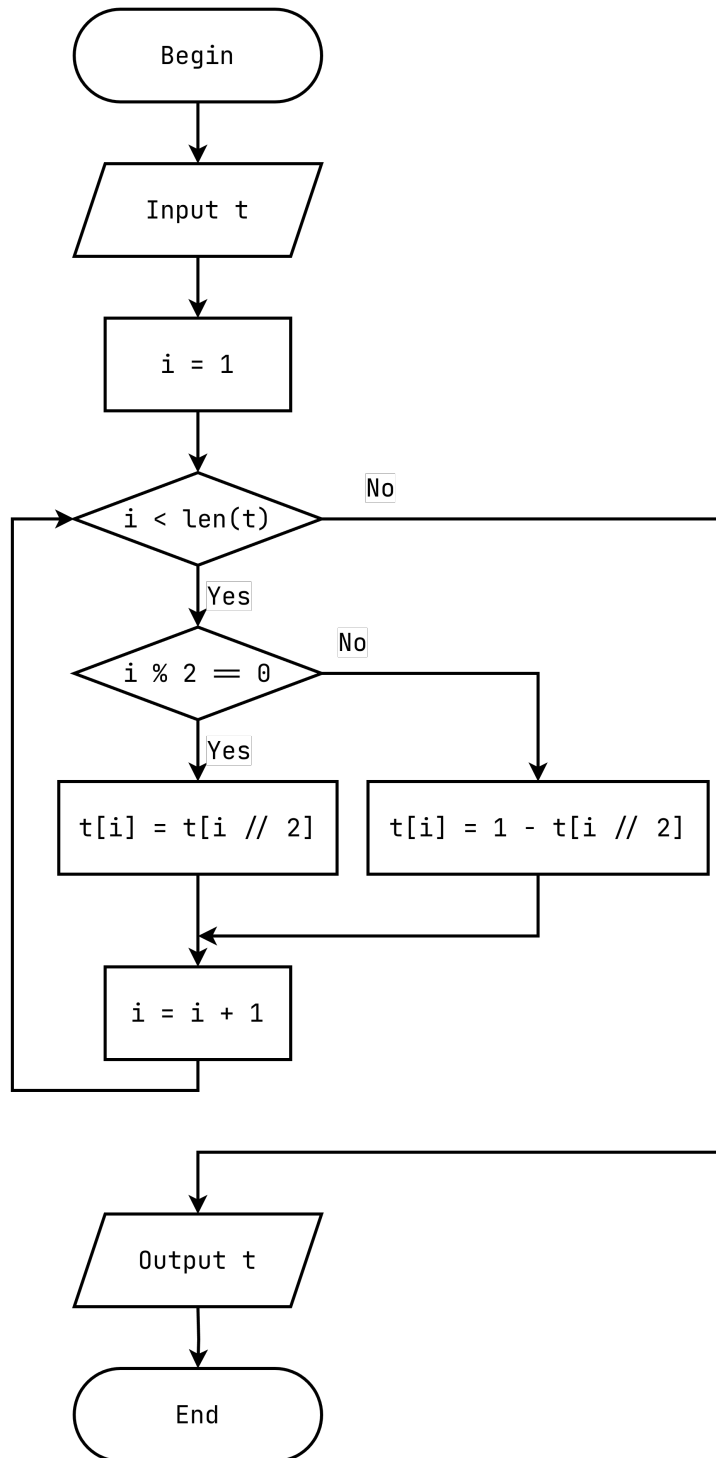
print(''.join([chr(k + ord('a') - 1) for k in key]))

```

Для варианта выше программа выдаёт ответ skjhbweryugqfptaczxd.

5. Алгоритмизация и программирование. Анализ алгоритма, заданного в виде блок-схемы (2 балла) [Спустя вечность]

Аксель любит строить разные последовательности, и вчера ему пришёл в голову алгоритм, который показан ниже в виде блок-схемы:



В качестве t Аксель вводит массив, содержащий битовую последовательность из 2^{32} нулей. Нумерация элементов массива начинается с нуля.

Определите, какая последовательность из 8 бит будет находиться, начиная с индекса 4294967124 (4294967124, 4294967125, ..., 4294967131). В ответ введите последовательность бит в порядке возрастания их индексов в последовательности без пробелов.

Пример ввода ответа:

01010101

Ответ: 10011001

Решение

Анализ алгоритма (это алгоритм генерации **последовательности Туэ-Морса**) показывает, что для вычисления элемента с индексом i не нужно вычислять все элементы до него. На самом деле, достаточно переписать этот алгоритм в виде рекурсивного:

```

def solve(i):
    if i == 0:
        return 0
    elif i % 2 == 0:

```

```

        return solve(i // 2)
    else:
        return 1 - solve(i // 2)

start = 4294967124
for s in range(start, start+8):
    print(solve(s))

```

Значение, возвращаемое функцией `solve` в базовом случае (при $i = 0$), меняется в зависимости от того, чем исходно заполнен массив t (0, если нулями, и 1, если единицами).

Если массив исходно заполнен нулями, а искомым фрагмент битовой последовательности начинается с индекса 4294967124, программа выдаёт ответ 10011001.

6. Телекоммуникационные технологии (3 балла)

[Управление перегрузкой]

Основная задача протокола TCP – обеспечение надежной доставки данных. Одна из возможных проблем – потеря сообщений на промежуточных устройствах в силу ошибок или переполнения памяти. Для предотвращения таких потерь используется **управление перегрузкой в TCP**.

В управлении перегрузкой отправитель регулирует скорость передачи данных, постепенно увеличивая ее (мы все же хотим передавать данные как можно быстрее) и уменьшая скорость передачи при возникновении ошибки. Существует множество алгоритмов управления перегрузкой. Одни из них – Tahoe. В нем используются следующие понятия:

- **RTT (Round-Trip Time)** — номинальное время полного пути пакета, то есть интервал между отправкой сегмента TCP и получением соответствующего ACK (подтверждения). Учтите, что это эталонное значение. В течение времени, равного RTT, отправитель может отправить большое количество пакетов, не дожидаясь ACK.
- **cwnd (Congestion Window)** — окно перегрузки, переменная на стороне отправителя, определяющая максимальное количество неподтверждённых сегментов, которое можно отправить в сеть без ACK. Этим окном как раз ограничивается скорость передачи. Чем больше окно, тем быстрее (интенсивнее) отправляются TCP-сегменты.
- **MSS (Maximum Segment Size)** — максимальный размер полезной нагрузки TCP-сегмента (обычно 1460 байт для Ethernet); используется как единица для роста cwnd.
- **ssthresh (Slow Start Threshold)** — пороговое значение окна перегрузки; разделяет фазы медленного старта (см. далее) и предотвращения перегрузки. В Tahoe изначально ssthresh не задаётся фиксированно (принимается равным бесконечности), а устанавливается динамически при первой потере. Будем считать (несколько упрощая), что при первой потере $ssthresh = cwnd/2$.
- **Slow Start** — фаза управления перегрузкой, где cwnd удваивается каждый RTT (при начальном значении, равном 1 MSS), чтобы быстро «нащупать» реальную пропускную способность сети.
- **Congestion Avoidance** — фаза, наступающая после достижения ssthresh. В этой фазе cwnd растёт линейно (+1 MSS за RTT), чтобы стабилизировать передачу без перегрузки.

Сначала скорость передачи растёт согласно Slow Start, а как случается первая потеря, рассчитывается ssthresh, после чего cwnd сбрасывается в 1 MSS, заново запускается Slow Start (cwnd удваивается каждый RTT), но уже до установленного ssthresh. Достигнув ssthresh, алгоритм переходит в режим Congestion Avoidance, при котором рост $cwnd + 1$ MSS за RTT, пока не произойдет новая потеря, после чего ssthresh уменьшается вдвое снова. Это создаёт «зубчатую» кривую cwnd.

Например, если потеря случилась при $cwnd=256$ MSS, то $ssthresh = cwnd/2=128$ MSS, cwnd сбрасывается в 1 MSS, возврат в Slow Start — cwnd удваивается каждый RTT до нового ssthresh (128 MSS). Достигнув ssthresh, переход в Congestion Avoidance — рост $cwnd + 1$ MSS за RTT, пока не произойдет новая потеря.

Пусть передается файл из 64 сегментов (по 1460 байт) передаётся по сети с $RTT = 120$ мс. Используется алгоритм TCP Tahoe.

- Slow Start: cwnd удваивается каждый RTT, начиная с 1 MSS.
- После потери: $ssthresh = cwnd / 2$, $cwnd = 1$ MSS.
- Достигнув ssthresh, переход к Congestion Avoidance (+1 MSS/RTT).
- Потеря происходит при $cwnd = 16$ MSS.

Найдите общее время передачи всех 64 сегментов в миллисекундах. Выберите наиболее близкий к полученному значению вариант ответа:

- 590
- 840
- 1150
- 1430
- 1710
- 1990

- 2160
- 2270
- 2400
- 2550

Ответ: 1430 (1440)

Решение

В этой задаче требуется аккуратно реализовать алгоритм, описанный в условии. Это можно сделать или вручную (что будет достаточно трудоёмко) или создать имитационную программную модель, например, такую:

```
SEGMENTS_COUNT = 64
RTT = 120
MAX_CWND = 16

cwnd = 1
sssthresh = None
segments_transferred = 0

rounds = 0

while segments_transferred < SEGMENTS_COUNT:
    if cwnd == MAX_CWND:
        sssthresh = cwnd // 2
        cwnd = 1

    segments_transferred += cwnd
    rounds += 1

    print(rounds, cwnd, segments_transferred, sssthresh, sep='\t')

    if sssthresh is None or cwnd < sssthresh:
        cwnd *= 2
    else:
        cwnd += 1

print(rounds * RTT)
```

Программа выдаёт 1440 миллисекунд, и ближайший ответ из имеющихся – 1430.

7. Технологии сортировки и фильтрации данных (3 балла)

[Расширяя круг общения]

Для анализа социальных сетей часто применяют понятие **социального графа**. В социальном графе пользователи являются вершинами, а каждое ребро показывает, что пользователи добавили друг друга в список друзей.

Исследователь собрал социальный граф для некоторых пользователей социальной сети Y. Этот граф хранится в виде одной таблицы `friends` со следующими полями:

- `first_user_id` — целое положительное число, идентификатор первого пользователя;
- `second_user_id` — целое положительное число, идентификатор второго пользователя.

При этом пара полей (`first_user_id`, `second_user_id`) является первичным ключом.

Каждая такая пара для удобства хранится дважды. Например, если пользователи с ID 1 и 2 добавили друг друга в список друзей, таблица будет содержать две записи:

<code>first_user_id</code>	<code>second_user_id</code>
1	2
2	1

Исследователь собирает данные аккуратно и даёт несколько гарантий:

- Дружба между двумя пользователями всегда хранится двумя строками;

- Нет ситуаций, когда пользователь добавил в список друзей самого себя.

Два различных пользователя знают друг друга **через одно рукопожатие**, если существует третий пользователь, отличающийся от них и являющийся им другом, но при этом сами пользователи не являются друзьями. Иначе говоря, в социальном графе между этими пользователями нет ребра, но есть путь через одну вершину. Например, если пользователи с ID 1 и 3 не добавляли друг друга в список друзей, но пользователь с ID 2 имеет в списке друзей этих пользователей, то пользователи с ID 1 и 3 знают друг друга через одно рукопожатие.

Определите, сколько различных пользователей знает пользователь с ID 10 через одно рукопожатие. Обратите внимание, что самого пользователя и его прямых друзей считать не нужно. В ответе введите одно целое неотрицательное число. Если пользователь с указанным ID не встречается в таблице, введите 0.

Пример ввода ответа:

17

Ответ: 5

Решение

Самый простой способ: найти всех пользователей, которые являются друзьями друзей нужного пользователя, и вычеркнуть из этого списка лишних (самого пользователя и его непосредственных друзей).

Получим друзей пользователя с ID 5:

```
Select second_user_id From friends  
Where first_user_id = 10;
```

Получим список: 9, 11, 181, 182, 183.

Дальше напишем запрос, который найдёт всех друзей пользователей из списка выше, не включая исходного пользователя:

```
Select friends.second_user_id From friends  
Where first_user_id In (9, 11, 181, 182, 183) And second_user_id <> 10;
```

Получим список из 5 элементов: 8, 12, 184, 85, 186.

Теперь необходимо убрать из ответа пересечение двух списков. В этом варианте пересечение пустое, но в других вариантах может встретиться несколько таких пользователей, которые влияют на ответ. Это относится, например, к пользователю с ID 5.

В итоге в этом варианте ответом будет 5.